

Buddy — Final Architecture & Implementation Plan (v3)

All questions resolved. Ready for implementation.

Product Summary

Buddy is a professional workspace overlay and productivity supervisor. When you start a work session, Buddy takes over your screen, organizes your apps into: virtual space including workspaces, focus monitoring, and uses ML to get smarter about your work patterns over time.

Core loop:

```
Schedule tasks → Start session → Buddy takes over screen →  
Work in monitored workspaces → ML learns your patterns → Future sessions get smarter
```

Tech Stack (Final)

Layer	Technology	Notes
Desktop Shell	Tauri v2 (Rust)	Single binary, native OS access
Frontend	Vue 3 + TypeScript + Vite	Composition API + <code><script setup></code>
State	Pinia	Vue official store
Backend	Rust (Tauri IPC commands)	All OS interaction, monitoring, DB
Database	SQLite via <code>rusqlite</code>	Per-user, in app data directory
ML Engine	Python (scikit-learn / LightGBM)	Spawned by Tauri for training, ONNX for inference

Layer	Technology	Notes
AI Chat	Optional BYOK (cloud API or local Ollama)	User-configured, not bundled
Monitoring	Rust native	Win32 (Windows) + D-Bus/AT-SPI (Linux GNOME/KDE)
URL Monitoring	Hybrid	Window title parsing + browser history DB polling
Fonts	Inter + Geist Mono	Premium, modern
Styling	CSS Variables + scoped CSS	HSL-based dark theme

Architecture Overview





Core Concepts (All Confirmed)

Workspaces

Workspaces are **free-form virtual desktops within Buddy's overlay** — NOT tied 1:1 to tasks.

- Users can create as many workspaces as they want
- Each workspace can hold any combination of **registered/allowed apps**
- During a session, only apps assigned to the session's tasks can be used across ALL workspaces
- Example: Workspace 1 has VS Code + Chrome, Workspace 2 has Terminal + File Manager — both valid if those apps are allowed for the session's tasks

```
Session: "Sprint Day" (tasks: coding + documentation)
Allowed apps: VS Code, Chrome (docs only), Terminal, Obsidian

Workspace 1: VS Code + Terminal      user's choice
Workspace 2: Chrome + Obsidian      user's choice
Workspace 3: VS Code + Chrome        also fine

Trying to open Discord → ■ BLOCKED (not in session's allowed apps)
```

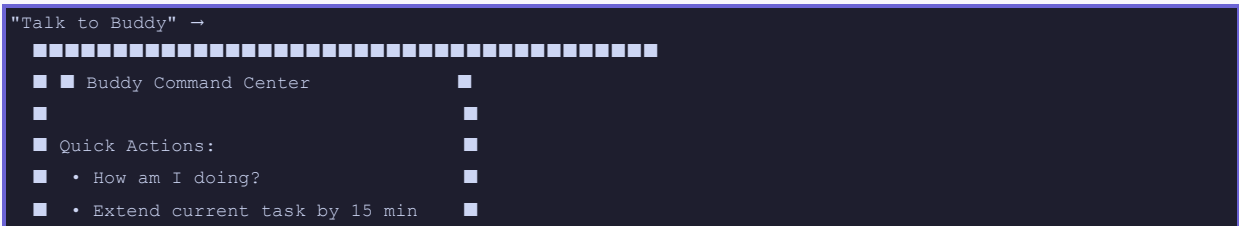
Boss Key (Emergency Exit)

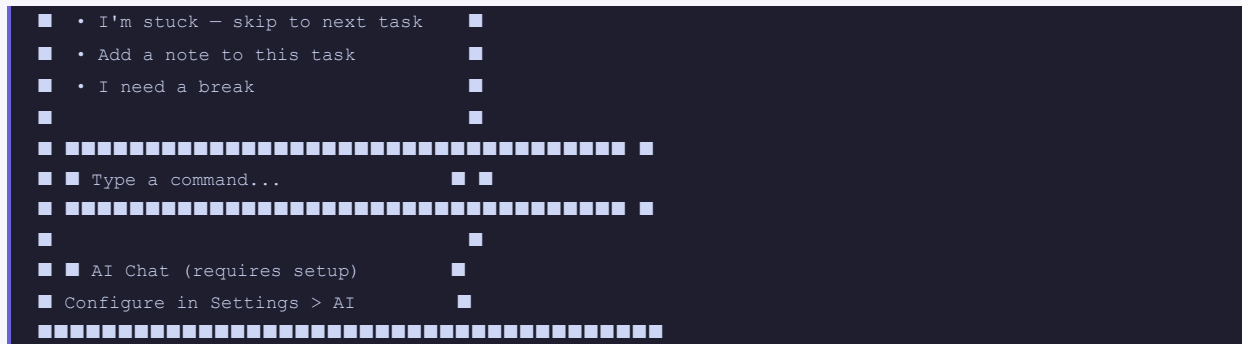
Use #	Behavior
1st use	Exit session. Logged. No penalty. Warning: "2 free exits remaining."
2nd use	Exit session. Logged. No penalty. Warning: "This is your last free exit."
3rd+ use	Exit session. Logged. Penalty applied. Focus score hit. Visible on dashboard.

The boss key is ALWAYS available — Buddy never truly traps you. But repeated exits have consequences.

Talk to Buddy

MVP: Command menu + smart templates (no AI required)

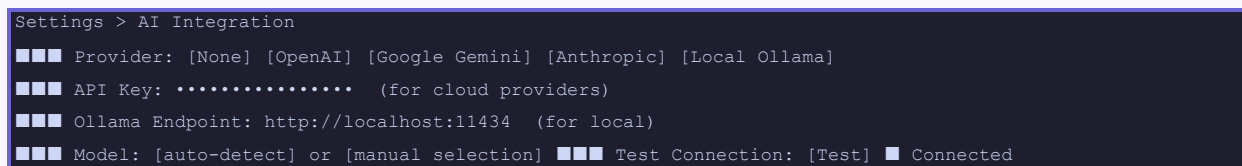




With AI Plugin enabled (optional BYOK):

- The command center becomes a conversational interface
- User can ask: "Break this task into subtasks", "Why do I always underestimate coding tasks?", "Suggest a better order for my remaining tasks"
- Powered by whichever AI the user configured (OpenAI, Gemini, Anthropic, or local Ollama)

AI Plugin (Settings > AI Integration)

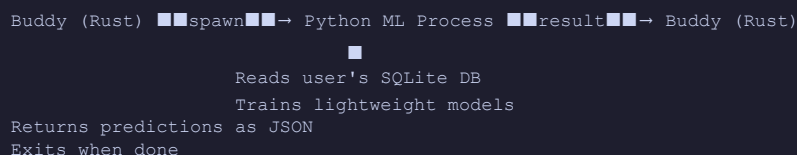


- When configured, "Talk to Buddy" upgrades from command menu to AI chat
- The AI has context: current session, task list, time remaining, violation history
- User can switch providers at any time

ML Engine — Making Buddy Smarter Over Time

Architecture

The ML runs as a **Python process spawned by Tauri on-demand** — not always running.



For **inference** (predictions during sessions), models are exported to **ONNX format** and loaded directly in Rust via `ort` crate — zero Python needed at runtime.

```
Training (occasional, Python):  
    SQLite data → scikit-learn/LightGBM → trained model → export to .onnx
```

```
Inference (real-time, Rust):  
User creates task → load .onnx model → predict duration → display estimate
```

ML Features(not finalized) *(reference for Sahil)*

Feature	Technique	Cold Start Fallback
Task duration estimation	Gradient boosting on past task completions (category, complexity, time-of-day, user history)	Rule-based defaults: coding=90m, writing=60m, design=120m, etc.
Feature	Technique	Cold Start Fallback
Optimal task ordering	Priority scoring: weighted sum of deadline proximity, estimated energy cost, past success rate at this time-of-day	Deadline-first + priority ordering
Burnout prediction	Time-series anomaly detection on input activity (typing speed drops, mouse movement decreases, longer pauses)	Fixed break intervals (25/5 pomodoro-style)
Productivity patterns	Statistical clustering: identify best hours, worst hours, optimal session length per user	Generic patterns ("most people focus best 9-11am")
Self-correcting estimates	Online learning: compare predicted vs actual duration, update model weights	Exponential moving average of past accuracy

Data Pipeline

```
Week 1-2: Rule-based defaults (no data yet)  
→ Buddy uses category-based heuristics  
→ Every completed task is logged to SQLite  
  
Week 3-4: First model training triggered (>20 completed tasks)  
→ Python spawns, trains on SQLite, exports .onnx  
→ Buddy starts using personalized estimates  
  
Month 2+: Model retrains weekly (or on-demand)  
→ Estimates get more accurate  
→ Burnout prediction and pattern detection activate  
→ Buddy visibly gets smarter
```

Training Data Schema (SQLite)

```
-- Every completed task becomes training data
CREATE TABLE ml_task_history (
    id INTEGER PRIMARY KEY,
    task_category TEXT,
    -- coding, writing, design, video, etc.
    estimated_mins INTEGER,
    -- what was predicted
    actual_mins INTEGER,
    -- how long it actually took
    time_of_day TEXT,
    -- morning/afternoon/evening/night
    day_of_week TEXT,
    -- monday-sunday
    energy_level INTEGER,
    -- inferred from input activity
    violations_during INTEGER,
    -- how distracted were they
    completed BOOLEAN,
    -- did they finish?
    created_at TIMESTAMP
);

-- Input activity for burnout
detection CREATE TABLE ml_activity_log (
    id INTEGER PRIMARY KEY,
    session_id INTEGER,
    timestamp
TIMESTAMP,
    keystrokes_per_min
FLOAT,
    mouse_distance_per_min
FLOAT,
    idle_seconds INTEGER,
    active_window TEXT
);
```

Monitoring System

Layers

Layer	What	How	Platform
1. Active Window	Track which app is in focus	Win32 <code>GetForegroundWindow</code> / GNOME D-Bus / KDE D-Bus / AT-SPI	Both
2. Process List	Detect new app launches, running processes	Win32 <code>CreateToolhelp32Snapshot</code> / <code>/proc</code> filesystem	Both
3. Browser URLs	Monitor which URLs are visited	Window title parsing (instant) + history DB polling (5-10s delay)	Both
4. Input Activity	Keyboard/mouse activity for idle/burnout detection	Win32 <code>GetLastInputInfo</code> / <code>libinput</code>	Both
5. Incognito Detection	Flag private browsing as suspicious	Detect "InPrivate"/"Incognito" in window title	Both

Incognito Handling

```
Browser window title contains "InPrivate" / "Incognito" / "Private Browsing"
→ Immediate violation alert: "■■ Private browsing detected during session"
→ Logged as a violation (cannot see URL, but the attempt is recorded)
```

→ Buddy doesn't try to see what's browsed – just flags the behavior → Repeated incognito violations escalate penalties

Violation Escalation

```
Level 1 – Unknown app opens:
  → Ask: "Spotify opened. Is this relevant to your tasks?"
  → User: [Allow for session] [Allow once] [Block]

Level 2 – Flagged app / off-task URL:
  → Visual warning: orange border flash + notification
  → Logged, 10-second grace period

Level 3 – Repeated violations (3+):
  → Auto-minimize the offending app →
  "3 violations – focus score dropping"

Level 4 – Blocked app / incognito detected:
  → Prevent launch or flag immediately
  → "This is blocked during sessions"
```

Music System

Built-in Ambient Player

Buddy ships with a small set of ambient sound loops:

- ■■■ Rain / Thunderstorm
- ■ Lo-fi beats (royalty-free)
- ■ Ocean waves
- ■ Fireplace
- ■ Café ambience
- ■ Forest / Nature
- ■ White / Pink / Brown noise

Lightweight audio files (~2-5 MB total) bundled with the app, played via the web audio API.

System Media Controls

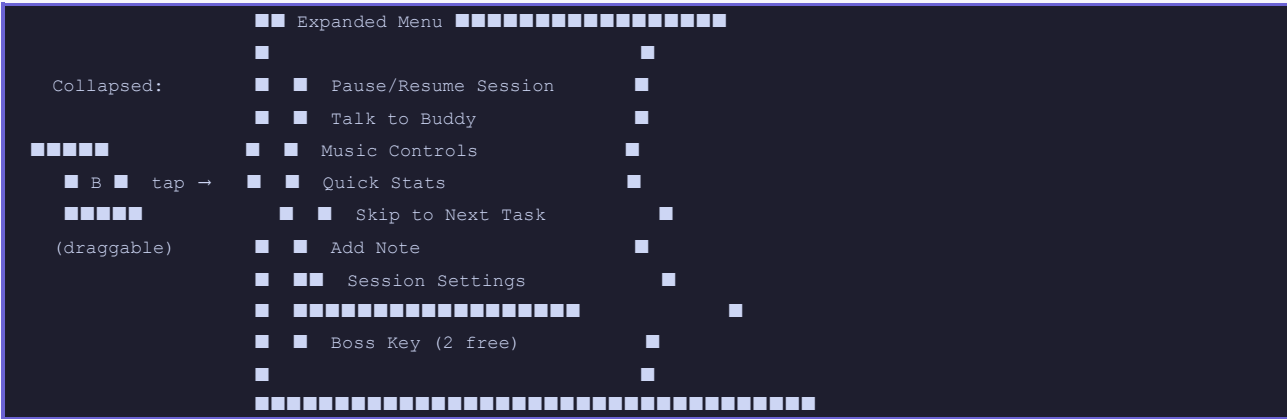
In addition to the built-in player, the assistive menu can control whatever media is playing on the system:

- Play / Pause (sends OS media key events)
- Next / Previous track
- Volume adjustment

Both are accessible from the assistive floating menu:

```
■ Music
■■■ Ambient: [Rain ■] [■ Play] [Vol ■■■●■■]
■■■ System: [■] [■] [■] [Vol ■■■■●■]
```

Assistive Floating Menu (Temporary Design)



- **Collapsed:** Small "B" bubble, semi-transparent, draggable along screen edges
- **Expanded:** Panel or radial menu with all controls
- **Boss Key:** Shows remaining free exits ("2 free" → "1 free" → "■■■ Penalty on exit")
- **Always on top** of workspace content during sessions

Default App Categories (Industry-Level)

Category	Default Apps (auto-detected)
Browser	Chrome, Firefox, Edge, Brave, Arc
Code Editor	VS Code, JetBrains (IntelliJ, PyCharm, WebStorm, CLion, etc.), Vim, Neovim, Sublime Text, Zed
Terminal	Windows Terminal, PowerShell, CMD, Konsole, Alacritty, kitty, WezTerm
Video Editing	DaVinci Resolve, Kdenlive, Shotcut, Blender (video mode), Premiere Pro, After Effects
Image / Design	Photoshop, GIMP, Krita, Inkscape, Figma (browser), Illustrator, Canva (browser)
3D / Animation	Blender, Maya, 3ds Max, FreeCAD, ZBrush, Cinema 4D
Audio	Audacity, REAPER, FL Studio, Ableton Live, Logic Pro, Ardour

Documents	LibreOffice, MS Word/Excel/PowerPoint, Google Docs (browser), PDF viewers
Notes	Obsidian, Notion (browser), Logseq, Joplin, OneNote
Communication	Slack, Discord, Teams, Zoom (allowed only when task-assigned)
File Management	File Explorer, Nautilus, Dolphin, Thunar, Nemo
Media Player	VLC, system default, Spotify, foobar2000
CAD / Engineering	AutoCAD, SolidWorks, KiCad, FreeCAD
Data / Science	Jupyter (browser), RStudio, MATLAB, Tableau
Custom	User-added applications

Apps are auto-detected from:

- **Windows:** Start Menu shortcuts, %ProgramFiles%, registry App Paths
- **Linux:** .desktop files in /usr/share/applications/ and ~/.local/share/applications/

Users can always add custom apps by browsing for the executable.

Database: Per-User Isolation

```
Windows: %APPDATA%\Buddy\data\buddy.db
Linux:   ~/.local/share/buddy/data/buddy.db
```

- Created fresh on first launch
- Schema migrations embedded in the Rust binary
- .gitignore ensures no DB files in the repo
- Single-user (no auth) — one DB per installation

Core Tables

```
-- Tasks (existing, evolved)
tasks, session_tasks, sessions, penalties, settings

-- New tables
workspaces                -- user-created workspace layouts
registered_apps            -- app registry (defaults + custom)
app_task_assignments       -- which apps are allowed per task
monitoring_events          -- real-time monitoring log violations
violation_records          -- violation records with escalation level
boss_key_usage             -- tracks boss key exits per session
ml_task_history            -- completed task data for ML training
ml_activity_log            -- input activity for burnout detection
ambient_sounds             -- user preferences for built-in audio
ai_config                 -- BYOK AI provider settings
```

Linux: GNOME/KDE Strategy (*only for Pushkar and Pranav for less confusions*)

GNOME

- **Window management:** Ship a tiny GNOME Shell Extension (auto-installed on first launch)
 - Exposes window list/titles via D-Bus
 - Allows show/hide/minimize commands
 - This is how major GNOME apps handle it (e.g., ApplIndicator, GSConnect)
- **Fallback:** AT-SPI for basic window detection if extension isn't installed

KDE Plasma

- **Window management:** KWin D-Bus scripting API
 - Inject small scripts via `org.kde.KWin` to enumerate/control windows
 - More open than GNOME — no extension needed
- **Fallback:** AT-SPI

Both

- **Process monitoring:** `/proc` filesystem (universal Linux)
- **Input activity:** `libinput` via Rust bindings
- **Browser history:** Same SQLite approach as Windows (different paths)

Design System □

Typography

- **Sans:** Inter (Google Fonts) — 9 weights, exceptional legibility
- **Mono:** Geist Mono (Vercel) — clean, minimal, modern

Color System (HSL-based dark theme)

```
:root {  
  /* Surfaces */  
  --bg-base: hsl(240, 10%, 5%);  
  --bg-surface: hsl(240, 6%, 9%);  
  --bg-elevated: hsl(240, 5%, 13%);  
  --bg-overlay: hsl(240, 4%, 16%);  
  
  /* Accent (blue-violet) */  
  --accent: hsl(245, 82%, 67%);  
  --accent-hover: hsl(245, 82%, 72%);  --accent-  
subtle: hsl(245, 82%, 67%, 0.12);  
  
  /* Text */  
  --text-primary: hsl(0, 0%, 95%);  
  --text-secondary: hsl(0, 0%, 65%);  --text-  
muted: hsl(0, 0%, 45%);  
  
  /* Status */  
  --success: hsl(142, 71%, 45%);  
  --warning: hsl(38, 92%,  
50%);  --danger: hsl(0, 84%,  
60%);  --info: hsl(199, 89%,  
48%); }
```

Animations

- Spring-based transitions for premium feel
- Smooth opacity fades for workspace switching
- Subtle scale + translate on hover for interactive elements
- 60fps target — no janky animations

MVP Screens (5 + Overlay)

1. Dashboard

- Live/last session stats, focus score trends
- Quick-start session button
- Weekly progress chart
- Upcoming tasks / deadlines

2. Task Canvas

- SVG mind-map (ported from Svelte → Vue 3)
- Hierarchical nodes, pan/zoom/drag, bezier arrows
- Task details panel (edit, assign apps)
- Per-task app assignment

3. Session Scheduler

- Select tasks → set available time → generate schedule
- ML-estimated durations (or rule-based fallback)
- Drag-to-reorder time blocks

- Break insertion (auto or manual)
- "Start Session" → transitions to Workspace

4. Settings

- General (theme, startup, shortcuts)
- App Integration (detected apps, custom apps, categories)
- AI Integration (BYOK provider config)
- Session defaults (break duration, penalty rules)
- Monitoring (depth, privacy, data retention)
- Music (ambient sound preferences, system media)

5. Monitoring Panel

- **Default:** "Buddy is watching ■■ · 0 violations · Focus: 94%"
- **Expanded:** Real-time process list, window tracking, URL log, input activity
- Session history with focus scores
- App usage breakdown

6. Workspace (The Overlay — CORE EXPERIENCE)

- Fullscreen takeover on session start
- Multiple free-form workspaces (switchable via workspace bar)
- Only allowed apps visible across all workspaces
- Assistive floating menu (B bubble)
- Session progress bar
- Violation notifications
- Boss key (2 free exits, then penalties)

Project Structure ▢

```
buddy/
■■■ src/                                # Vue 3 frontend
■   ■■■ main.ts
■   ■■■ App.vue
■   ■■■ assets/
■   ■   ■■■ styles/
■   ■   ■   ■■■ variables.css          # Design tokens
■   ■   ■   ■■■ base.css              # Reset + Inter/Geist Mono
■   ■   ■   ■■■ transitions.css       # Spring animations
■   ■   ■■■ audio/                   # Ambient sound loops
■   ■       ■■■ rain.ogg
■   ■       ■■■ lofi.ogg
■   ■       ■■■ ocean.ogg
■   ■       ■■■ fireplace.ogg
■   ■       ■■■ cafe.ogg
■   ■       ■■■ forest.ogg
■   ■       ■■■ whitenoise.ogg
■   ■■■ components/
■   ■   ■■■ common/                  # Button, Input, Card, Modal, Toast
■   ■   ■■■ dashboard/              # Dashboard widgets
■   ■   ■■■ canvas/                  # SVG mind-map components
■   ■   ■■■ session/                 # Session scheduler components
■   ■   ■■■ workspace/               # Workspace overlay
■   ■       ■■■ WorkspaceBar.vue      # Workspace tabs/switcher
■   ■       ■■■ AssistiveMenu.vue     # Floating B bubble + expanded menu
■   ■       ■■■ SessionBar.vue        # Progress bar + task info
■   ■       ■■■ ViolationToast.vue    # Violation notifications
■   ■       ■■■ BuddyCommand.vue      # "Talk to Buddy" command center
■   ■   ■■■ monitor/                 # Monitoring panel
■   ■   ■■■ settings/                # Settings panels
■   ■       ■■■ GeneralSettings.vue
■   ■       ■■■ AppIntegration.vue
■   ■       ■■■ AIIntegration.vue     # BYOK provider config
■   ■       ■■■ SessionDefaults.vue
■   ■       ■■■ MonitoringSettings.vue
```

□

```
■ ■ ■ ■ ■ MusicSettings.vue
■ ■ ■ ■ ■ music/                # Ambient player + system controls
■ ■ ■ ■ ■   AmbientPlayer.vue
■ ■ ■ ■ ■   MediaControls.vue
■ ■ ■ ■ ■ composables/
■ ■ ■ ■ ■   useTauri.ts          # Tauri IPC helpers
■ ■ ■ ■ ■   useMonitor.ts       # Real-time monitoring state
■ ■ ■ ■ ■   useSession.ts       # Active session state
■ ■ ■ ■ ■   useWorkspace.ts     # Multi-workspace management
■ ■ ■ ■ ■   useAudio.ts         # Ambient player + system media
■ ■ ■ ■ ■   useAI.ts            # AI plugin communication
■ ■ ■ ■ ■ stores/
■ ■ ■ ■ ■   tasks.ts
■ ■ ■ ■ ■   session.ts
■ ■ ■ ■ ■   workspace.ts
■ ■ ■ ■ ■   apps.ts
■ ■ ■ ■ ■   monitor.ts
■ ■ ■ ■ ■   settings.ts
■ ■ ■ ■ ■   ai.ts
■ ■ ■ ■ ■ views/
■ ■ ■ ■ ■   DashboardView.vue
■ ■ ■ ■ ■   TaskCanvasView.vue
■ ■ ■ ■ ■   SessionView.vue
■ ■ ■ ■ ■   WorkspaceView.vue   # The fullscreen overlay
■ ■ ■ ■ ■   MonitorView.vue
■ ■ ■ ■ ■   SettingsView.vue
■ ■ ■ ■ ■ router/
■ ■ ■ ■ ■   index.ts
■ ■ ■ ■ ■ types/
■ ■ ■ ■ ■   task.ts
■ ■ ■ ■ ■   session.ts
■ ■ ■ ■ ■   app.ts
■ ■ ■ ■ ■   workspace.ts
■ ■ ■ ■ ■   ai.ts
■ ■ ■ ■ ■ src-tauri/            # Rust backend
```

□

```
■ ■■■ src/
■ ■ ■■■ main.rs
■ ■ ■■■ lib.rs
■ ■ ■■■ commands/
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ tasks.rs
■ ■ ■ ■■■ sessions.rs
■ ■ ■ ■■■ apps.rs
■ ■ ■ ■■■ monitor.rs
■ ■ ■ ■■■ workspace.rs
■ ■ ■ ■■■ music.rs           # System media key events
■ ■ ■ ■■■ ai.rs             # AI plugin proxy (BYOK)
■ ■ ■■■ db/
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ models.rs
■ ■ ■ ■■■ migrations/
■ ■ ■ ■■■ queries.rs
■ ■ ■■■ monitor/
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ engine.rs         # Main monitoring loop
■ ■ ■ ■■■ violations.rs     # Detection + escalation
■ ■ ■ ■■■ input.rs          # Keyboard/mouse activity
■ ■ ■ ■■■ browser.rs        # URL monitoring (title + history DB)
■ ■ ■■■ apps/
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ registry.rs
■ ■ ■ ■■■ defaults.rs       # Industry-level defaults
■ ■ ■ ■■■ scanner.rs        # Auto-detect installed apps
■ ■ ■ ■■■ matcher.rs        # Process → registered app matching
■ ■ ■■■ window/
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ overlay.rs        # Fullscreen takeover
■ ■ ■ ■■■ workspace.rs      # Multi-workspace window management
■ ■ ■ ■■■ focus_trap.rs     # Focus trapping + boss key
■ ■ ■■■ session/
```

□

```

■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ scheduler.rs      # Algorithmic scheduling
■ ■ ■ ■■■ tracker.rs        # Execution tracking
■ ■ ■ ■■■ boss_key.rs       # Boss key usage tracking + penalties
■ ■ ■■■ ml/
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ inference.rs      # ONNX model loading + prediction
■ ■ ■ ■■■ fallback.rs       # Rule-based defaults (cold start)
■ ■ ■ ■■■ trainer.rs        # Spawn Python training process
■ ■ ■■■ ai/
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ plugin.rs         # BYOK provider management
■ ■ ■ ■■■ openai.rs         # OpenAI API client
■ ■ ■ ■■■ gemini.rs         # Google Gemini API client
■ ■ ■ ■■■ anthropic.rs      # Anthropic API client
■ ■ ■ ■■■ ollama.rs         # Local Ollama client
■ ■ ■■■ os/
■ ■ ■ ■■■ mod.rs           # Trait definitions
■ ■ ■ ■■■ windows.rs        # Win32 API implementations
■ ■ ■ ■■■ linux.rs         # GNOME D-Bus + KDE D-Bus + AT-SPI
■ ■■■ Cargo.toml
■ ■■■ tauri.conf.json
■■■ ml/                    # Python ML scripts
■ ■■■ requirements.txt      # scikit-learn, lightgbm, onnx
■ ■■■ train_estimator.py     # Task duration estimation model
■ ■■■ train_burnout.py       # Burnout prediction model
■ ■■■ train_patterns.py      # Productivity pattern analysis
■ ■■■ export_onnx.py         # Export trained models to ONNX
■■■ package.json
■■■ vite.config.ts
■■■ tsconfig.json
■■■ .gitignore              # *.db, *.sqlite3, *.onnx, node_modules, target

```

Phased Build Order

Phase 1 — Foundation (Week 1-2)

- ☐ Initialize Tauri v2 + Vue 3 + TypeScript + Vite project (fresh start)
- ☐ Rust module structure (all mod.rs files, trait definitions)
- ☐ Design system: CSS variables, Inter + Geist Mono, dark theme
- ☐ SQLite setup (rusqlite, per-user path, initial migrations)
- ☐ Task CRUD commands in Rust (port from Python backend)
- ☐ Vue routing + shell layout (sidebar, topbar, view stubs)
- ☐ Pinia store skeletons
- ☐ .gitignore cleanup (ensure DB, build artifacts excluded)

Phase 2 — Task Canvas (Week 2-3)

- ☐ Port SVG mind-map from Svelte → Vue 3
- ☐ Parent-child hierarchy, pan/zoom/drag, bezier arrows
- ☐ Task detail panel (edit properties)
- ☐ Per-task app assignment UI
- ☐ Connect to Rust backend via Tauri IPC

Phase 3 — App Registry & Scanner (Week 3-4)

- ☐ App registry in Rust (model, industry-level defaults)
- ☐ App scanner: auto-detect installed apps (Windows + Linux)
- ☐ Settings > App Integration UI
- ☐ Process matcher (running processes → registered apps)
- ☐ Full Settings page (all sections)
- ☐ Settings > AI Integration UI (BYOK config)

Phase 4 — Monitoring Engine (Week 4-5)

- ☐ OS abstraction layer (traits + Windows impl + Linux GNOME/KDE impl)
- ☐ Active window tracking
- ☐ Process list monitoring
- ☐ Browser URL monitoring (title parsing + history DB polling)
- ☐ Incognito detection
- ☐ Input activity detection (keyboard/mouse)
- ☐ Monitoring panel UI (simplified + advanced views)
- ☐ Real-time event emission (Tauri events → Vue)

Phase 5 — Session Engine (Week 5-6)

- ☐ Session scheduler in Rust (algorithmic, from mate's spec)
- ☐ Rule-based task estimation (cold start fallback)
- ☐ Session scheduler UI (task selection, time blocks, drag-to-reorder)
- ☐ Break insertion logic
- ☐ Session execution tracker (active task, time, progress)
- ☐ Boss key tracking (2 free, then penalties)
- ☐ Session history + statistics storage

Phase 6 — Workspace Overlay (Week 6-7)

- ☐ Fullscreen takeover (always-on-top, focus trap)
- ☐ Multi-workspace system (free-form, switchable)
- ☐ Workspace bar UI (tabs, add/remove/switch)
- ☐ Window show/hide per workspace (only allowed apps)
- ☐ Assistive floating menu (B bubble, all controls)
- ☐ Talk to Buddy command center (command menu + templates)
- ☐ Music system (ambient player + system media controls)
- ☐ Violation detection → escalation pipeline
- ☐ Session bar (progress, current task, time)

Phase 7 — ML Integration (Week 7-8)

- ☐ Python training scripts (estimator, burnout, patterns)
- ☐ ONNX export pipeline
- ☐ Rust ONNX inference (ort crate)
- ☐ ML data collection (log completed tasks, activity)
- ☐ Cold start → trained model transition logic ML-
- ☐ powered estimates in session scheduler

Phase 8 — Dashboard + Polish (Week 8-9)

- ☐ Dashboard with live data (focus score, streaks, stats)
 - ☐ Charts (weekly progress, app usage, ML insights)
 - ☐ AI plugin integration (BYOK cloud + Ollama)
 - ☐ Cross-platform testing (Windows + Linux GNOME + Linux KDE)
 - ☐ Performance optimization (monitoring loop, DB queries, animations)
 - ☐ Bug fixes + edge cases
 - ☐ Ambient audio asset integration
-

What's NOT in MVP (Post-MVP Backlog)

- ☐ Android companion app + Supabase sync
- ☐ Advanced penalty tiers (actual OS-level app blocking)

- ☐ Calendar page with external calendar integration
- ☐ Profile / account system
- ☐ Hyprland/Sway (wlroots) Linux support
- ☐ Browser extension (optional enhancement over history DB)
- ☐ Multi-user support
- ☐ Export / import data
- ☐ Plugin system for third-party integrations
- ☐ LLM-powered session engine (upgrade from traditional ML)

□